

Machine Learning – Week 2 Assignment

By Gaurav Naudiyal – Msc Augmented & Virtual Reality

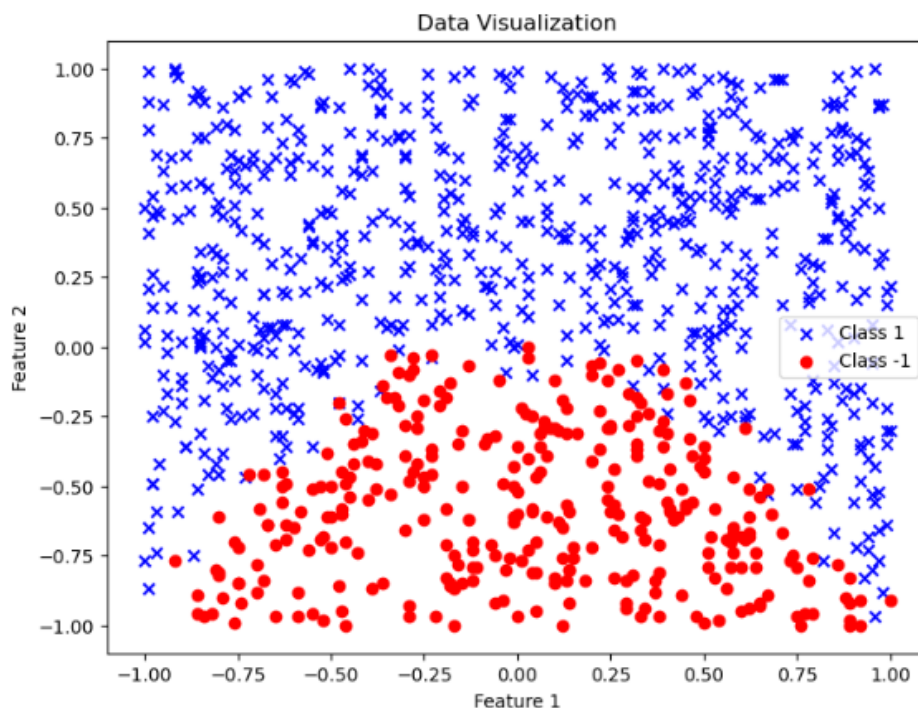
Dataset identifier :

The first line of my data set is : # id:19-19-19

Question (a) Logistic Regression

Data Visualization :

Firstly, I loaded the dataset into python using pandas and separated X1 & X2 and y. Then I plotted the data using matplotlib to see how the classes are spread over.



In the plot :

Blue x's show data points in class 1 whereas red o's show data points in class -1. There seem to be a clear separation between the two data points in the y axis (feature 2) although there is a little near the center.

Logistics Regression Model

Here I used LogisticRegression to fit a model into the data. The logisticregression model is used to find decision boundary that separates the two classes. Here were the results:

Coefficients : `[[0.21146321 5.34410595]]`

Intercept : `[1.74907327]`

The coefficients define the relationship between the independent & the dependent variable. On the other hand Intercept is used to show the mean value of the response variable when all the predictor variables in the model are equal to zero.

Model Interpretation :

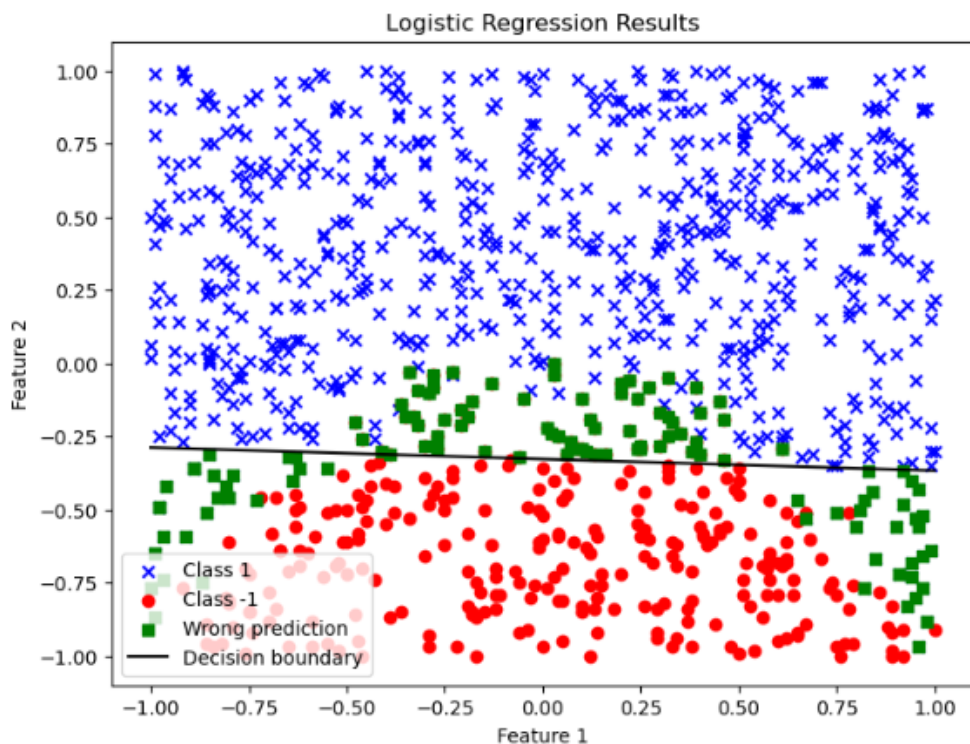
Feature influence:

A higher absolute coefficient means the feature has more influence. In the dataset the coefficient of feature 2 (5.34410595) is much more larger than feature 1 (0.21146321) which means that feature 2 has a greater influence in the prediction.

If the coefficient is positive, increasing that feature increases the likelihood of predicting class 1 & viceversa. In our case both the coefficients are positive so increasing either makes it more likely the model will predict class 1.

Prediction Results

I used the train model to predict the class for each data point and plotted actual classes, wrong predictions (green squares) and the model's decision boundary line (black line).



Decision boundary:

It represents the line where the model is likely to assign a plot to either class. The boundary line was calculated using the equation $(\text{coef}_1 * x) + (\text{coef}_2 * y) + \text{intercept} = 0$.

This was solved for y i.e. $y_{\text{vals}} = -(\text{coef}[0] * x_{\text{vals}} + \text{intercept}) / \text{coef}[1]$ to plot the line.

Observations :

The logistic regression classifier correctly classifies most points but there are some errors which are represented by green squares especially near the boundary between the classes.

According to the coefficients, feature 2 is much more important than feature 1 and has a greater influence in the prediction. This is also proved by the decision boundary being horizontal instead of vertical.

Appendix : Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline

df = pd.read_csv('week2.csv')
print(df.head())

X1 = df.iloc[:,0]
X2 = df.iloc[:,1]
y = df.iloc[:,2]
X = np.column_stack([X1, X2])

plt.figure(figsize=(8,6))
plt.scatter(X1[y==1], X2[y==1], marker='x', color='blue', label='Class 1')
plt.scatter(X1[y==-1], X2[y==-1], marker='o', color='red', label='Class -1')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.title('Data Visualization')
plt.show()

from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X, y)
print('Model coefficients:', model.coef_)
print('Model intercept:', model.intercept_)

y_pred = model.predict(X)
plt.figure(figsize=(8,6))
plt.scatter(X1[y==1], X2[y==1], marker='x', color='blue', label='Class 1')
plt.scatter(X1[y==-1], X2[y==-1], marker='o', color='red', label='Class -1')
plt.scatter(X1[y_pred!=y], X2[y_pred!=y], marker='s', color='green', label='Wrong
prediction')

# Decision boundary
coef = model.coef_[0]
intercept = model.intercept_[0]
x_vals = np.linspace(X1.min(), X1.max(), 100)
```

```

y_vals = -(coef[0]*x_vals + intercept)/coef[1]

plt.plot(x_vals, y_vals, color='black', label='Decision boundary')

plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

plt.legend()

plt.title('Logistic Regression Results')

plt.show()

```

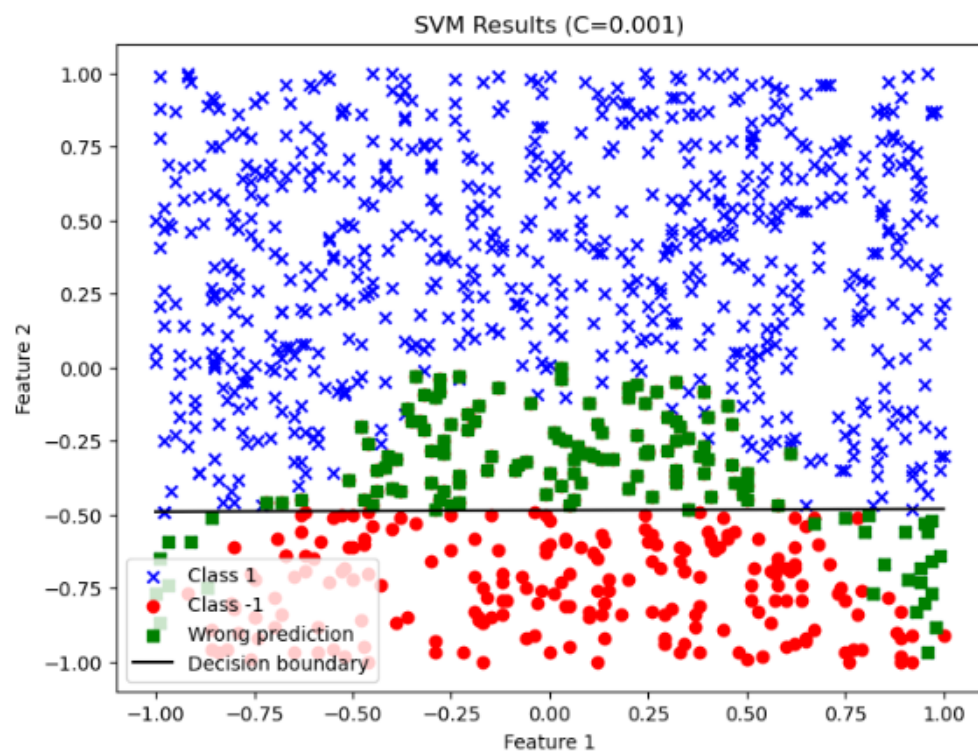
Question (b) SVM:

Method:

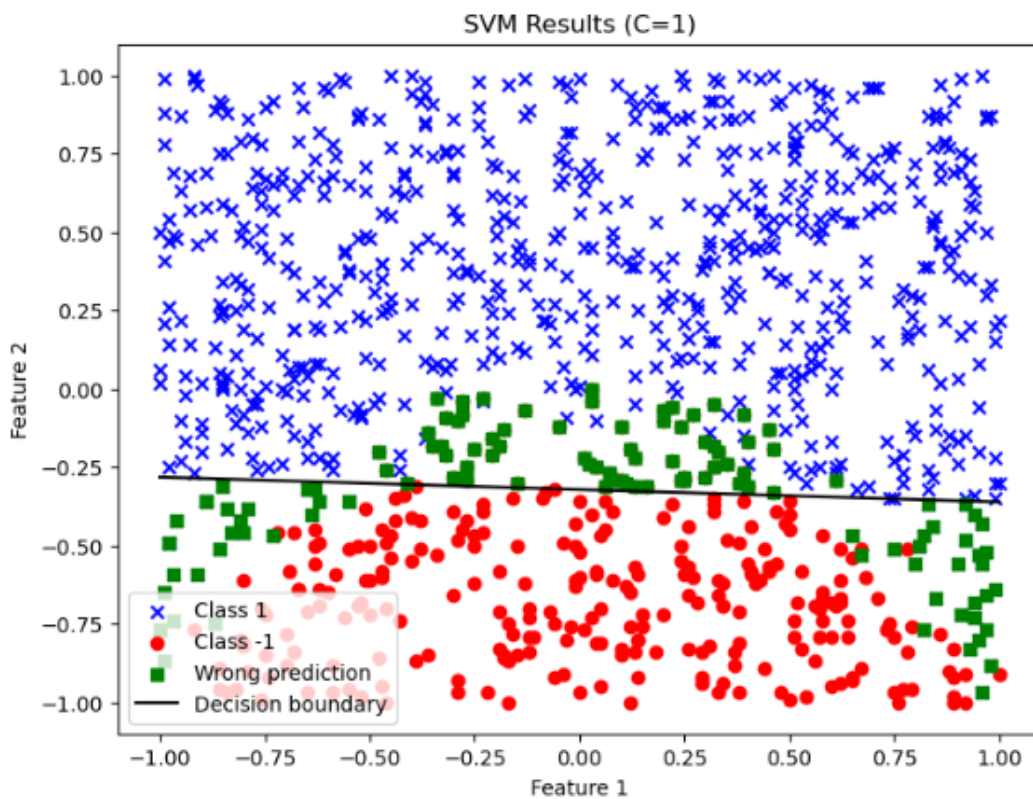
Using LinearSVC, I trained three SVM Models with different values of regularization parameter C : 0.001, 1 and 100. A regularization parameter controls the strength of regularization in machine learning models.

Plots :

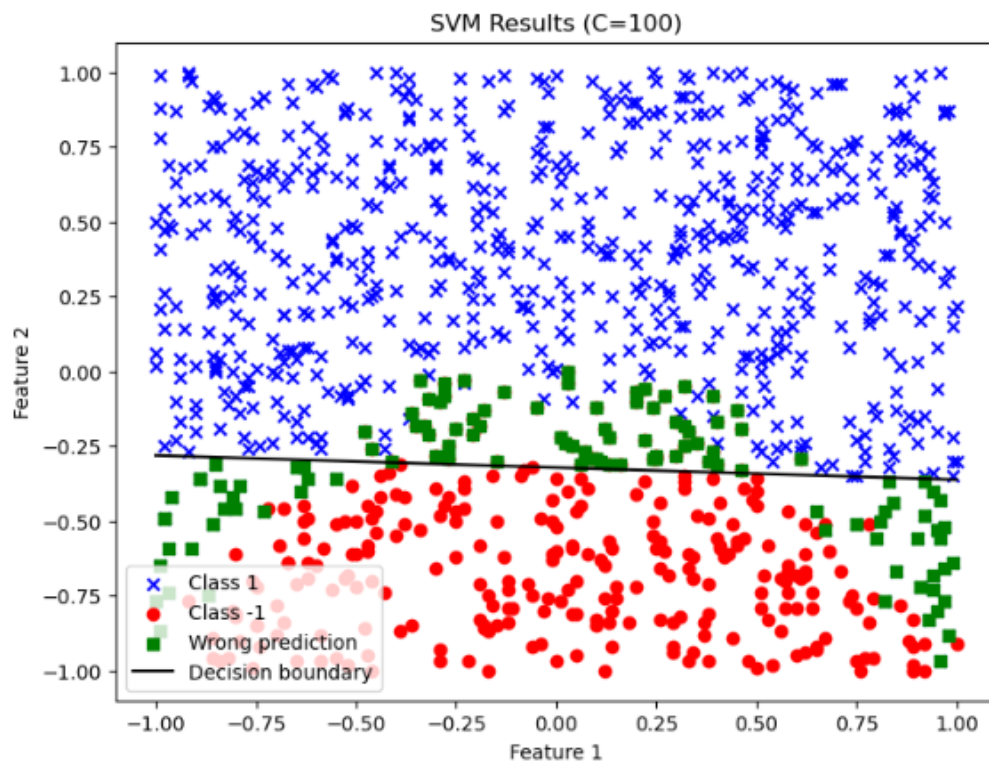
For SVM C = 0.001 the plot was :



For SVM $C = 1$ the plot was :



For SVM $C = 100$ the plot was :



For each plot:

X axis = Feature 1

Y axis = Feature 2

Blue x's = Class 1

Red o's = Class -1

Green squares = Wrong Prediction

Black line = Decision boundary

Model parameters :

SVM (C=0.001): coef = $\begin{bmatrix} -0.00236389 & 0.47868761 \end{bmatrix}$ intercept = $[0.23246842]$

SVM (C=1): coef = $\begin{bmatrix} 0.07773562 & 1.96445837 \end{bmatrix}$ intercept = $[0.63140823]$

SVM (C=100): coef = $\begin{bmatrix} 0.08064668 & 1.9908474 \end{bmatrix}$ intercept = $[0.64048386]$

Analysis of C parameter:

C (0.001) : The decision boundary (black line) is almost horizontal and a large number of green squares (misclassifications) are near the boundary.

C(1) : The decision boundary is less flat than C 0.001 but still heavily y-axis influenced it has fewer misclassifications than C = 0.001 but overfitting.

C(100) : This boundary line shifts the data more closely which leads to lesser green squares near the boundary.

In all the three cases, most misclassifications are near the decision boundary, where features lap between classes.

Comparison to Logistic Regression :

SVM and logistic regression produce similar boundaries but SVM makes fewer mistakes at higher C(100) at the cost of overfitting.

The number of green squares changes with the value of C.

Appendix (Code) :

```
from sklearn.svm import LinearSVC
svm1 = LinearSVC(C=0.001, max_iter=10000)

svm1.fit(X, y)

svm2 = LinearSVC(C=1, max_iter=10000)

svm2.fit(X, y)

svm3 = LinearSVC(C=100, max_iter=10000)

svm3.fit(X, y)
print('SVM (C=0.001): coef =', svm1.coef_, 'intercept =', svm1.intercept_)

print('SVM (C=1): coef =', svm2.coef_, 'intercept =', svm2.intercept_)

print('SVM (C=100): coef =', svm3.coef_, 'intercept =', svm3.intercept_)
y_pred1 = svm1.predict(X)
y_pred2 = svm2.predict(X)
y_pred3 = svm3.predict(X)
def plot_svm_results(X1, X2, y, y_pred, model, title):
    plt.figure(figsize=(8,6))

    plt.scatter(X1[y==1], X2[y==1], marker='x', color='blue', label='Class 1')

    plt.scatter(X1[y==-1], X2[y==-1], marker='o', color='red', label='Class -1')

    plt.scatter(X1[y_pred!=y], X2[y_pred!=y], marker='s', color='green',
label='Wrong prediction')

    coef = model.coef_[0]

    intercept = model.intercept_[0]

    x_vals = np.linspace(X1.min(), X1.max(), 100)

    y_vals = -(coef[0] * x_vals + intercept) / coef[1]

    plt.plot(x_vals, y_vals, color='black', label='Decision boundary')

    plt.xlabel('Feature 1')

    plt.ylabel('Feature 2')

    plt.legend()

    plt.title(title)

    plt.show()

plot_svm_results(X1, X2, y, y_pred1, svm1, 'SVM Results (C=0.001)')

plot_svm_results(X1, X2, y, y_pred2, svm2, 'SVM Results (C=1)')

plot_svm_results(X1, X2, y, y_pred3, svm3, 'SVM Results (C=100)')
```

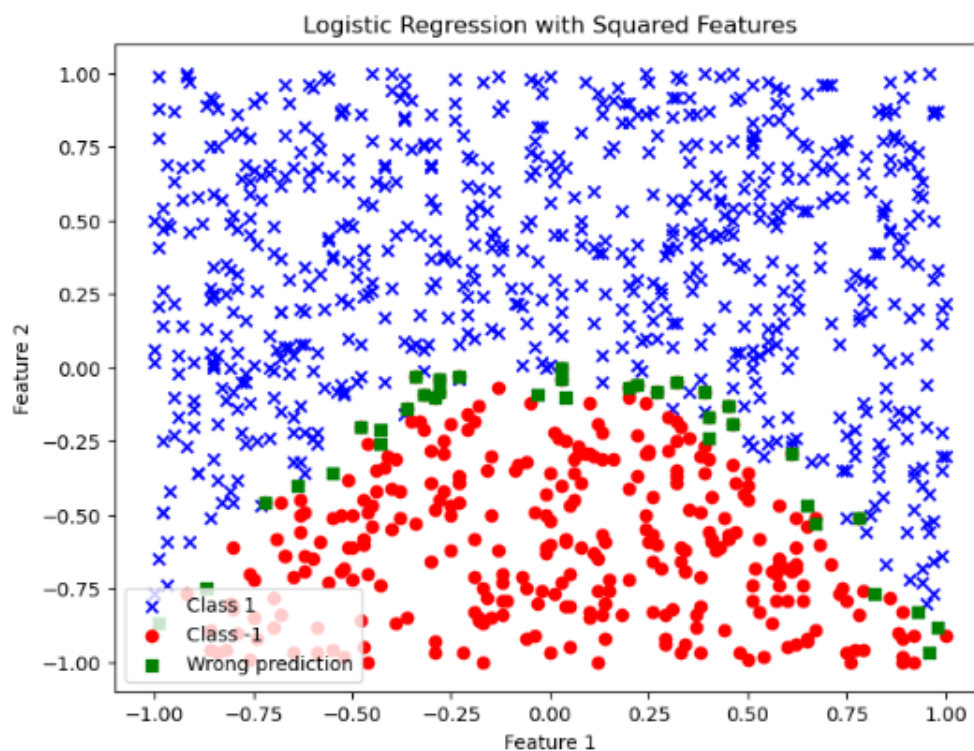

Question (c) Logistic Regression with Squared Features:

Approach :

I added 2 new features $X1^1$ and $X2^2$ to the dataset giving four features in total.

Results :

The plot below shows the predictions of the new model :



Blue x's are true Class 1 points whereas Red o's are true Class -1 points. Once again the green squares represent misclassified points.

Baseline classifier :

A baseline classifier, which predicts the most common class for every point is 0.68 which is shown in the image below:

```
Baseline accuracy: 0.6826826826826827
```

This means that if you always guess the majority class, you'd get about 68% of the predictions correct.

Comparison from LVM & Logistic Regression :

After adding squared features, the model can capture more complex relationships between the features and the target class.

The new model improves separation compared to linear regression approach.

Comparing the overall results, **adding non-linear features helps the model perform better than simply guessing majority classes.**

Appending : Code:

```
X1_sq = X1 ** 2
X2_sq = X2 ** 2

X_new = np.column_stack([X1, X2, X1_sq, X2_sq])
model_new = LogisticRegression()

model_new.fit(X_new, y)

print('Model coefficients:', model_new.coef_)

print('Model intercept:', model_new.intercept_)
y_pred_new = model_new.predict(X_new)
plt.figure(figsize=(8,6))

plt.scatter(X1[y==1], X2[y==1], marker='x', color='blue', label='Class 1')

plt.scatter(X1[y==-1], X2[y==-1], marker='o', color='red', label='Class -1')

plt.scatter(X1[y_pred_new!=y], X2[y_pred_new!=y], marker='s', color='green',
label='Wrong prediction')

plt.xlabel('Feature 1')

plt.ylabel('Feature 2')

plt.legend()

plt.title('Logistic Regression with Squared Features')
```

```
plt.show()
baseline = np.sign(np.sum(y))

baseline_pred = np.full_like(y, baseline)

baseline_accuracy = np.mean(baseline_pred == y)

print('Baseline accuracy:', baseline_accuracy)
```